CrimeScope LA

Interactive Exploration of Reported Crime Patterns in Los Angeles

CSC 220 — Advanced Programming | Sample Software Project Proposal | November 2026

Executive summary and motivation

We propose CrimeScope LA, a Java desktop application that transforms historical Los Angeles reported-crime records into an interactive exploration tool. Large public datasets can be difficult to interpret in raw tabular form. Our application will let users inspect the dataset, select a research question, adjust filters, view computed summaries and charts, and export their current results to CSV. The purpose is descriptive analysis of recorded incidents—not crime prediction or a judgment about neighborhood safety.

Dataset and initial assessment

The proposed source is the supplied Los Angeles crime_data.csv file. Initial inspection identifies 1,004,894 records, 28 columns, 21 distinct police-area names, and 140 distinct offense descriptions. The file includes occurrence and report dates, police area, offense description, time of occurrence, premises description, and other fields. Occurrence-year labels include 2020 (199,847), 2021 (209,876), 2022 (235,259), 2023 (232,345), 2024 (127,567). Date coverage and missing or invalid values will be verified before comparing periods.

The application will report total and unique rows, duplicate counts, column names and inferred types, missing-value summaries, and frequencies of analysis-relevant categorical fields. Any ambiguous codes, missing premises, or partial-year coverage will be documented.

Research questions

RQ1 — Top-k offense categories

Which offense descriptions occur most frequently for a user-selected police area and date range? Users choose area, period, and k; results show ranked counts and a bar chart.

RQ2 — Change over time

How do counts of a selected offense change by month or year across a selected period? The application calculates absolute and percentage changes where comparable periods are available and presents a line chart.

RQ3 — Day and hour patterns

How do recorded occurrence times vary by weekday and hour for a selected offense or police area? The application groups incidents into time intervals and visualizes the distribution.

RQ4 — Comparison across areas

How does the composition of reported offense categories differ between selected police areas? The application compares counts and within-area proportions; it will not equate raw counts with population-adjusted crime rates.

RQ5 — Premises associated with offenses

Which premises descriptions are most frequent for a selected offense category and period? Users can request top-k premises types and compare their proportions among records with known premises.

Initial descriptive visualization

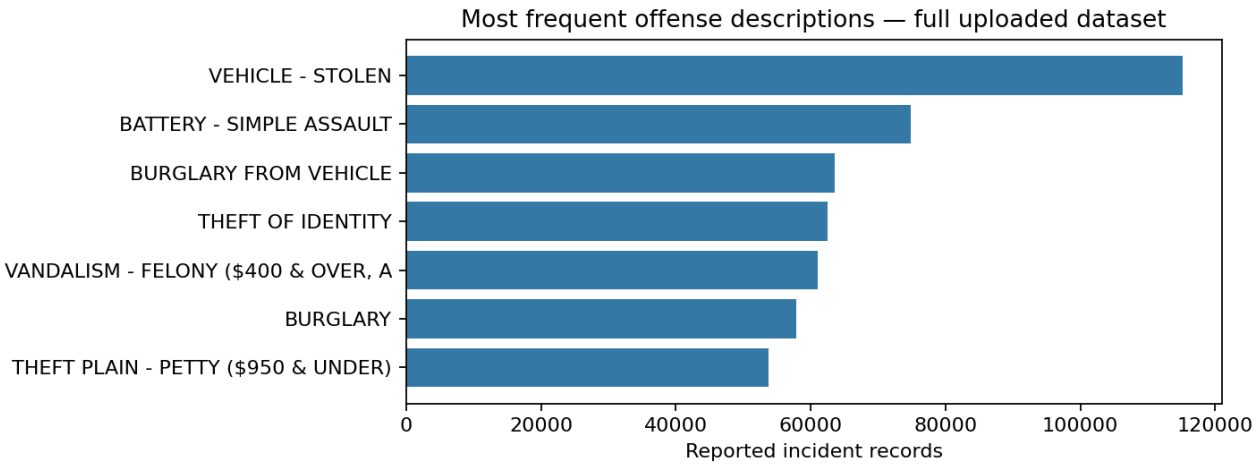


Figure 1. Counts calculated from the supplied CSV. These are incident-record frequencies, not population-adjusted crime rates.

Proposed interactive application

The interface will provide a dataset overview; a research-question selector; controls for area, offense, period, k, or other applicable parameters; Run Analysis and Clear buttons; a results table; an embedded chart; and Export CSV. Controls will change according to the selected question. A separate chart window is acceptable if integration within the main window is impractical.



Figure 2. Proposed interface wireframe. Values are illustrative of the full uploaded dataset; the finished Java application will compute results dynamically.

Implementation and software design

We propose Java with Swing or JavaFX and a charting library such as XChart or JFreeChart. Candidate components include CrimeRecord (data model), CrimeCsvReader (input parsing), DatasetProfiler (overview and validation), CrimeAnalysisService (filters and aggregations), ChartBuilder (visualization), CsvResultExporter (CSV writing), and CrimeScopeWindow (user interactions). These are provisional responsibilities, not a mandatory architectural pattern. The design will emphasize single responsibility, separation of concerns, encapsulation, KISS, and DRY.

For performance, the application will parse the file once, keep only necessary attributes, and reuse suitable collections and precomputed groupings where useful. Initial development may use a smaller representative subset, followed by tests on the complete file.

Testing and quality assurance

JUnit 5 unit tests will cover at least five scenarios across two or more non-GUI operations, including frequency counts, top-k ordering and ties, filters, empty inputs, and period-change calculations. We will validate date parsing, missing fields, invalid parameters, and CSV escaping. Automated GUI or integration tests are not proposed.

Results export and documentation

Users will be able to save the current computed results as a CSV file using a GUI file chooser. Exported columns will identify the selected measure and values, rather than reproduce the raw dataset. The repository will include Java code, tests, a README with build and execution instructions, the proposal, updated class diagram, and a final report documenting findings and limitations. A reproducible Maven build or runnable JAR will be prepared near completion.

Development plan and responsibilities

Nov 10: confirm data and research questions, proposal, work allocation, and GitHub repository. Nov 12: demonstrate initial parsing or analytical code. Nov 17: implement core calculations and basic GUI. Nov 19: demonstrate integration and an initial chart. Nov 20–Dec 2: complete analysis, CSV export, tests, documentation, and packaging. Dec 3: submit public GitHub repository. Final demonstration and individual technical interviews: date to be announced.

For a two-person team, Developer A will lead data ingestion, profiling, and analytical operations; Developer B will lead GUI, visualization, and CSV export. Both developers will review each other's work, contribute tests and documentation, and understand the full implementation.

Limitations and ethical interpretation

These records reflect reported incidents, reporting practices, and data quality. They do not capture all offenses, prove causes, or directly measure an individual's risk. Missing values, reporting delays, inconsistent classifications, and incomplete time coverage may affect comparisons. We will avoid identifying victims and will not infer safety rankings from raw area totals.

Source and external assistance

Primary dataset: user-supplied crime_data.csv (Los Angeles reported-crime records). The original publication URL and applicable terms will be verified and cited in the final report. Documentation, code examples, and AI assistance will be acknowledged when used; the project's design decisions, testing, and explanations remain the team's responsibility.